# 3.1 Inheritance and generalization

Chapter 19 begins by moving shared properties and behavior into a superclass. A car **is a** vehicle; a boat **is a** vehicle. A car **has an** engine. The first relationship can justify inheritance; the second suggests an object property, or composition.

## Find the shared members

We already know how to create a class. Now imagine copying its properties and methods into three similar classes. When a rule changes, we would have to fix every copy. A superclass gives that shared rule one home.

Page 358 in our textbook shows a `Vehicle` superclass with `Car` and `Boat` subclasses. Both subclasses have four properties, three that were inherited from the superclass and one that is unique to each subclass.

## Declare and load the hierarchy

Create a `Class3` folder inside the `src/Tutorials` folder. Save this as `Vehicle.php` inside that new folder:

```php
<?php
declare(strict_types=1);

class Vehicle
{
    public string $makeModel = 'Unknown';

    public function getLabel(): string
    {
        return $this->makeModel;
    }
}
```

Save this as `src/Tutorials/Class3/Car.php`:

```php
<?php
declare(strict_types=1);

class Car extends Vehicle
{
    public string $bodyShape = 'Hatchback';
}
```

Save this as `src/Tutorials/Class3/Boat.php`:

```php
<?php
declare(strict_types=1);
```

```
class Boat extends Vehicle
{
    public string $countryOfRegistration = 'US';
}
```

Save this as `public/tutorials/3-1-inheritance-and-polymorphism.php`:

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/../../src/Tutorials/Class3/Vehicle.php';
require_once __DIR__ . '/../../src/Tutorials/Class3/Car.php';
require_once __DIR__ . '/../../src/Tutorials/Class3/Boat.php';

$car = new Car();
$car->makeModel = 'City Compact';
$boat = new Boat();
$boat->makeModel = 'Harbor Cruiser';

echo $car->getLabel(), ' / ', $car->bodyShape, '<br>', PHP_EOL;
echo $boat->getLabel(), ' / ', $boat->countryOfRegistration, '<br>', PHP_EOL;
```

Run `php public/tutorials/3-1-inheritance-and-polymorphism.php` from the repo or open it in your browser:

```
City Compact / Hatchback
Harbor Cruiser / US
```

Neither subclass defines `getLabel()`. Both inherit it. `$this` inside that inherited method is the particular Car or Boat receiving the call. We created two objects, not a child object plus a separate parent object.

The parent declaration must be available when PHP loads the child. With explicit `require_once`, load Vehicle first. Autoloading will automate this later with Chapter 20. These public properties make inheritance visible; the next tutorial refines their visibility.

## Add a level

Save this as `src/Tutorials/Class3/SailBoat.php` and require it after Boat in the entry script:

```
<?php
declare(strict_types=1);

class SailBoat extends Boat
{
```

```
    public int $numberOfMasts = 2;
}
```

Append this to `public/tutorials/3-1-inheritance-and-polymorphism.php`:

```
$sailBoat = new SailBoat();
$sailBoat->makeModel = 'Wind Runner';
echo $sailBoat->getLabel(), ' / ', $sailBoat->numberOfMasts, '<br>', PHP_EOL;
var_dump($sailBoat instanceof Boat, $sailBoat instanceof Vehicle);
```

The added output is `Wind Runner / 2`, followed by two `bool(true)` values. `instanceof` asks whether this object belongs to a class or one of its subclasses. SailBoat inherits through Boat to Vehicle, but still has only **one direct parent**. PHP does not allow `extends Boat, Vehicle`.